MODULE 4

STATISTICAL THINKING

Detailed Study Notes • Distributions • Probability • CDFs • Statistical Models

Syllabus covered

- Distributions, histograms, outliers, summaries, variance, and reporting results
- Probability mass functions (PMFs), visualizations, class size paradox, and DataFrame indexing
- Cumulative distribution functions (CDFs), percentiles, random numbers, and percentile ranks
- Modeling distributions: exponential, normal, and lognormal distributions

Beginner-friendly concepts, formulas, worked examples, Python code, revision tables, and practice questions.

How to use these notes

This module connects descriptive statistics with probability models. Read each definition, follow the small numerical examples, and run the Python snippets to see how the ideas work.

Install and import

```
python3 -m pip install numpy pandas matplotlib scipy
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

If you are using a notebook, plots often appear under the cell. In a regular Python script, use `plt.show()` to display them.

Learning outcomes

• Describe a dataset with center, spread, shape, and unusual values.

• Explain PMFs, CDFs, percentiles, and random sampling.

• Use Pandas indexing to select rows and columns correctly.

• Recognize exponential, normal, and lognormal distributions.

• Summarize and report statistical results clearly.

1. What is statistical thinking?

Statistical thinking means using data to understand variation, uncertainty, patterns, and differences. Instead of focusing on one observation, we ask what the whole dataset shows and how reliable that conclusion may be.

Population, sample, and variable

• **Population:** the full group we want to understand, such as all students in a university.

• **Sample:** the subset actually observed, such as 100 selected students.

• **Variable:** a characteristic recorded for each unit, such as height, marks, or travel time.

• **Variation:** the fact that observations differ from one another.

Example: If we want to understand the average time students take to reach college, we may survey a sample. The sample average estimates the population average, but it may differ because of sampling variation.

2. Distributions

A **distribution** describes the values a variable takes and how often or how likely those values are. A distribution can be described by its center, spread, shape, and unusual observations.

2.1 Center

• **Mean:** sum of values divided by the number of values. It uses every value and can be affected by extreme observations.

• **Median:** the middle value after sorting; for an even number of values, average the two middle values.

• **Mode:** the most frequent value or category.

```
data = [2, 3, 3, 4, 18]
print(np.mean(data))    # 6.0
print(np.median(data))  # 3.0
```

The value 18 pulls the mean upward, while the median stays near the central observations.

2.2 Spread

• **Range:** maximum minus minimum.

• **Interquartile range (IQR):** third quartile Q3 minus first quartile Q1; describes the spread of the middle half.

• **Variance:** average squared distance from the mean, using a population or sample formula depending on the question.

• **Standard deviation:** square root of variance; expressed in the same units as the data.

2.3 Shape

• **Symmetric:** the two sides are roughly similar.

• **Right-skewed:** a long tail extends toward larger values; mean is often greater than median.

• **Left-skewed:** a long tail extends toward smaller values; mean is often less than median.

• **Unimodal / bimodal:** one prominent peak / two prominent peaks.

• **Uniform-like:** values occur at roughly similar rates across a range.

2.4 Plotting a histogram

A histogram groups numerical values into bins and shows the count or density in each interval. It is useful for seeing shape, clusters, gaps, skewness, and possible outliers.

```
marks = [45, 50, 52, 55, 58, 60, 61, 65, 70, 72, 80, 92]

plt.hist(marks, bins=5, edgecolor="black")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.title("Distribution of marks")
plt.show()
```

Bin choice matters: too few bins can hide patterns; too many can make random variation look important. Try a few sensible bin counts.

2.5 Other distribution visualizations

• **Box plot:** shows median, quartiles, spread, and potential outliers.

• **Density plot / KDE:** smooth estimate of a distribution's shape.

• **ECDF:** shows the fraction of observations less than or equal to each value.

• **Dot plot:** shows individual values and is especially useful for small datasets.

3. Outliers

An **outlier** is an observation unusually far from the rest of the data. It may be a recording error, a rare but valid event, or evidence of a different subgroup. Do not delete it automatically.

3.1 IQR rule

Calculate Q1 (25th percentile), Q3 (75th percentile), and IQR = Q3 – Q1. A common flagging rule is:

Lower fence = Q1 – 1.5 × IQR; **Upper fence** = Q3 + 1.5 × IQR.

Values outside the fences are flagged as potential outliers. This is a diagnostic rule, not proof that a value is wrong.

```
data = np.array([10, 11, 12, 12, 13, 14, 15, 60])
q1, q3 = np.percentile(data, [25, 75])
iqr = q3 - q1
low = q1 - 1.5 * iqr
high = q3 + 1.5 * iqr
outliers = data[(data < low) | (data > high)]
print(q1, q3, iqr, low, high)
print("Potential outliers:", outliers)
```

3.2 What to do with an outlier

• Check units, data entry, duplicate records, and sensor or measurement problems.

• Compare the value with source records when possible.

• Ask whether it is plausible in the real context.

• Report how it affects the mean, median, spread, or model.

• Keep valid extreme values unless there is a clear reason to exclude them; explain any exclusions.

4. Summarizing distributions

A good summary usually describes **center, spread, shape, and unusual observations**. Choose statistics that match the shape of the data.

4.1 Common summary statistics

Statistic	Meaning / formula	Use
Mean	Sum of values ÷ n	Roughly symmetric data without extreme outliers
Median	Middle sorted value	Skewed data or data with outliers
Range	Maximum – minimum	Very simple overall spread
Q1 / Q3	25th / 75th percentiles	Position and spread
IQR	Q3 – Q1	Robust spread of the middle 50%
Variance	Mean squared deviation from mean (with convention)	Spread in squared units
Standard deviation	Square root of variance	Spread in original units

4.2 Example summary in Python

```
data = pd.Series([10, 11, 12, 12, 13, 14, 15, 60])
print(data.describe())
print("Variance:", data.var())       # sample variance (ddof=1)
print("Std dev:", data.std())        # sample standard deviation
```

Pandas `var()` and `std()` use sample formulas by default (degrees of freedom = 1). NumPy's `np.var()` and `np.std()` default to population-style division by n. Select the convention that matches the question.

4.3 Reporting results

A statistical report should state the sample size, units, useful summary statistics, and the pattern shown by the plot. Avoid claiming more than the data supports.

Example: “For the 8 recorded values, the median was 12.5 and the distribution included one unusually large value, 60. Because the data are right-skewed, the median and IQR may describe the typical value and spread more robustly than the mean and standard deviation.”

5. Variance

Variance measures how far values spread around their mean. It averages squared deviations, so larger deviations contribute disproportionately and the result is in squared units.

5.1 Population and sample variance

For a population of N values: **$\sigma^2 = \Sigma(x_i - \mu)^2 / N$**. For a sample of n values, the commonly used unbiased estimator is **$s^2 = \Sigma(x_i - \bar{x})^2 / (n - 1)$**.

The sample formula divides by n – 1 because the sample mean is estimated from the same data. This correction helps estimate population variance without systematic downward bias under standard sampling assumptions.

```
values = [2, 4, 6]
print(np.var(values))          # population variance = 2.666...
print(np.var(values, ddof=1))  # sample variance = 4.0
```

5.2 Hand calculation

For values 2, 4, 6: mean = 4. Deviations are –2, 0, 2; squared deviations are 4, 0, 4. Their sum is 8. Population variance = 8/3 ≈ 2.67; sample variance = 8/2 = 4.

Interpretation: variance is non-negative; zero variance means every value is identical. Standard deviation is often easier to interpret because it returns to the original units.

6. Probability mass functions (PMFs)

A **probability mass function** gives the probability of each possible value of a discrete random variable. For a variable X, p(x) = P(X = x).

6.1 PMF rules

• Each probability is between 0 and 1.

• The probabilities over all possible values sum to 1.

• A PMF applies to discrete outcomes such as number of heads, number of arrivals, or a die result.

6.2 Example: fair die

For a fair six-sided die, P(X = x) = 1/6 for x = 1, 2, 3, 4, 5, 6.

```
x = np.arange(1, 7)
pmf = np.ones(6) / 6

plt.bar(x, pmf, width=0.8, edgecolor="black")
plt.xticks(x)
plt.xlabel("Die result")
plt.ylabel("Probability")
plt.title("PMF of a fair die")
plt.show()
```

6.3 Plotting a PMF from observed data

```
outcomes = pd.Series([1, 2, 1, 3, 2, 1, 4, 2, 3, 1])
pmf = outcomes.value_counts(normalize=True).sort_index()
pmf.plot(kind="bar")
plt.xlabel("Outcome")
plt.ylabel("Estimated probability")
plt.title("Empirical PMF")
plt.show()
```

The empirical PMF estimates probabilities from relative frequencies. A sample estimate may differ from the true probability, especially for small samples.

6.4 Limits of PMFs

A PMF is for discrete variables. A continuous variable such as height or waiting time does not usually assign positive probability to each exact value; instead, probabilities are described over intervals using a probability density function (PDF). For a continuous variable, $P(a \le X \le b)$ is the area under the density curve from a to b.

6.5 Other visualizations

• PMF bar chart for discrete probabilities.

• Histogram for sample observations of numeric values.

• PDF / density curve for continuous models.

• CDF or empirical CDF for cumulative probabilities.

• Box plot for compact summaries of center, spread, and potential outliers.

7. The class size paradox

The **class size paradox** is a sampling/averaging effect: a student selected at random is more likely to be in a large class than a small class. When asking about the class attended by a randomly selected student, large classes contribute more students and therefore receive more weight.

7.1 Simple example

Suppose one class has 10 students and another has 30 students. If you choose a class uniformly at random, each class has a 50% chance of selection and the average class size is (10 + 30)/2 = 20. If you choose a student uniformly at random, the chance of coming from the larger class is 30/40 = 75%.

The expected class size experienced by a randomly chosen student is (10×10 + 30×30)/(10+30) = 25, which is larger than the ordinary average of 20.

7.2 Why it matters

• Different sampling methods answer different questions.

• An average across groups is not always the same as the average experienced by individuals.

• Similar weighting issues occur in hospitals, schools, workplaces, and online platforms.

8. DataFrame indexing

Pandas indexing selects rows and columns. The two most important indexers are `loc` (label-based) and `iloc` (integer-position-based).

```
df = pd.DataFrame({
    "name": ["Asha", "Bala", "Charan"],
    "marks": [82, 75, 91],
    "city": ["Ongole", "Tirupati", "Nellore"]
}, index=["s1", "s2", "s3"])

print(df["marks"])             # one column (Series)
print(df.loc["s1"])            # row with label s1
print(df.loc["s1", "marks"])   # value at label s1, column marks
print(df.iloc[0])              # first row by position
print(df.iloc[0, 1])           # first row, second column
print(df.loc["s1":"s2"])       # label slice includes s2
print(df.iloc[0:2])             # positional slice excludes position 2
```

8.1 Boolean indexing

```
high_marks = df[df["marks"] >= 80]
print(high_marks)

selected = df.loc[df["marks"] >= 80, ["name", "marks"]]
print(selected)
```

8.2 Common indexing rules

• `df['column']` selects one column.

• `df.loc[row_label, column_label]` uses labels.

• `df.iloc[row_position, column_position]` uses integer positions.

• `df.loc[condition, columns]` filters rows and chooses columns.

• Use `.copy()` if you need an independent DataFrame to modify.

• Label slices with `loc` include the ending label; positional slices with `iloc` exclude the stop position.

9. Cumulative distribution functions (CDFs)

A **cumulative distribution function** gives the probability that a random variable is less than or equal to a value: $F(x) = P(X \le x)$. It accumulates probability from the smallest values upward.

9.1 CDF properties

• F(x) is between 0 and 1.

• F(x) never decreases as x increases.

• For sufficiently small x, F(x) approaches 0; for sufficiently large x, it approaches 1.

• For a discrete variable, the CDF rises in steps; for a continuous variable, it is often smooth.

9.2 Example: die CDF

```
x = np.arange(1, 7)
cdf = np.cumsum(np.ones(6) / 6)

plt.step(x, cdf, where="post")
plt.scatter(x, cdf)
plt.ylim(0, 1.05)
plt.xlabel("Die result x")
plt.ylabel("P(X <= x)")
```

```
plt.title("CDF of a fair die")
plt.show()
```

For a fair die, F(3) = P(X ≤ 3) = 3/6 = 0.5.

9.3 Empirical CDF

For observed values x■, …, x■, the empirical CDF at x is the fraction of observations less than or equal to x.

```
data = np.array([2, 3, 3, 5, 8])
sorted_data = np.sort(data)
y = np.arange(1, len(data) + 1) / len(data)

plt.step(sorted_data, y, where="post")
plt.xlabel("Value")
plt.ylabel("Fraction <= x")
plt.title("Empirical CDF")
plt.ylim(0, 1.05)
plt.show()
```

9.4 CDF versus PMF

PMF	CDF
Probability at each exact discrete value: P(X = x).	Cumulative probability up to x: P(X ≤ x).
Values sum to 1 across all discrete outcomes.	Values rise from 0 toward 1 and never decrease.
Often shown with bars or stems.	Often shown as a step function for discrete data.

10. Percentile-based statistics

The p-th percentile is a value at or below which approximately p% of observations fall. The 50th percentile is the median; the 25th and 75th percentiles are Q1 and Q3.

```
data = [12, 15, 18, 20, 22, 25, 30, 35]
print(np.percentile(data, 25))
print(np.percentile(data, 50))
print(np.percentile(data, 75))
```

Different software can use different interpolation conventions for percentiles, so tiny differences may occur for small datasets.

10.1 Percentile rank

A percentile rank describes the percentage of observations at or below a given score. If a score is at the 90th percentile, it is higher than or equal to roughly 90% of the reference observations, depending on the convention.

10.2 Comparing percentile ranks

Raw scores may not be comparable when groups have different distributions. A percentile rank gives a relative position within a chosen reference group, but it does not say how large the absolute difference is.

Example: A score of 80 could be at the 90th percentile in one exam and the 70th percentile in a more difficult exam. Percentiles depend on the comparison population and assessment.

10.3 Percentiles in Python

```
scores = np.array([55, 60, 62, 70, 75, 80, 90])
p90 = np.percentile(scores, 90)
print("90th percentile:", p90)

# Approximate fraction at or below a chosen score
score = 75
percentile_rank = 100 * np.mean(scores <= score)
print("Percentile rank:", percentile_rank)
```

11. Random numbers and simulation

Random numbers help model uncertainty, test algorithms, and simulate outcomes. A pseudo-random number generator produces a sequence that behaves like random data but can be reproduced using a fixed seed.

```
rng = np.random.default_rng(42)
uniform_values = rng.random(5)       # values in [0, 1)
die_rolls = rng.integers(1, 7, size=10)
normal_values = rng.normal(loc=0, scale=1, size=1000)

print(uniform_values)
print(die_rolls)
```

11.1 Common random generators

• `rng.random(n)`: n values uniformly sampled from 0 inclusive to 1 exclusive.

• `rng.integers(low, high, size=n)`: random integers; high is excluded.

• `rng.normal(loc, scale, size)`: normal samples with mean loc and standard deviation scale.

• `rng.exponential(scale, size)`: exponential samples where scale is the mean (1/rate).

• `rng.lognormal(mean, sigma, size)`: samples whose natural logarithms are normal.

11.2 Reproducibility

Set a seed when you want the same random sequence to appear again, for example while teaching, debugging, or reproducing an experiment. A seed does not make the data truly random; it makes the pseudo-random sequence repeatable.

12. Modeling distributions

A distribution model is a mathematical description of how values occur. A model can help estimate probabilities, simulate data, and compare observed patterns with an expected pattern. Always check whether the model is reasonable for the data and context.

12.1 Normal distribution

The **normal distribution** is a symmetric, bell-shaped continuous distribution defined by mean μ and standard deviation σ. The mean, median, and mode are equal for an ideal normal distribution.

The empirical 68–95–99.7 rule says that for a normal distribution, about 68% of values lie within 1σ of the mean, 95% within 2σ, and 99.7% within 3σ.

```
rng = np.random.default_rng(1)
x = rng.normal(loc=50, scale=10, size=2000)
plt.hist(x, bins=30, density=True, alpha=0.6, edgecolor="white")
plt.title("Simulated normal distribution")
plt.xlabel("Value")
plt.ylabel("Density")
plt.show()
```

12.2 Z-score

A z-score expresses how many standard deviations a value is from the mean: **z = (x** – μ**) /** σ. A positive z-score is above the mean; a negative z-score is below it.

```
x, mu, sigma = 70, 60, 5
z = (x - mu) / sigma
print(z)  # 2.0 standard deviations above the mean
```

12.3 Exponential distribution

The **exponential distribution** models non-negative waiting times between events in a constant-rate Poisson process. It is right-skewed and has its highest density near zero.

With rate λ > 0, the density is f(x) = λe^(–λx) for x ≥ 0, and the mean is 1/λ. In NumPy, the `scale` parameter is 1/λ, which equals the mean.

```
rng = np.random.default_rng(2)
waiting = rng.exponential(scale=5, size=2000)  # mean 5
plt.hist(waiting, bins=30, density=True, alpha=0.6)
plt.title("Simulated exponential waiting times")
plt.xlabel("Waiting time")
plt.ylabel("Density")
plt.show()
```

12.4 Lognormal distribution

A **lognormal distribution** is a positive-valued distribution where the natural logarithm of the variable follows a normal distribution. It is often strongly right-skewed and can model quantities formed by multiplicative growth, such as some income, size, or duration data.

If ln(X) is normal with parameters μ and σ, then X is lognormal. In NumPy, `rng.lognormal(mean=μ, sigma=σ)` uses μ and σ for the underlying normal logarithm, not the arithmetic mean and standard deviation of X.

```
rng = np.random.default_rng(3)
sizes = rng.lognormal(mean=2, sigma=0.5, size=2000)
plt.hist(sizes, bins=30, density=True, alpha=0.6)
plt.title("Simulated lognormal values")
plt.xlabel("Value")
plt.ylabel("Density")
plt.show()
```

12.5 Compare the three distributions

Distribution	Shape / support	Example use
Normal	Symmetric; any real value	Measurement variation, errors, approximately bell-shaped values
Exponential	Right-skewed; $x \geq 0$	Waiting time between events at a constant rate
Lognormal	Right-skewed; $x > 0$	Positive quantities with multiplicative variation

12.6 How to assess a model

• Plot the observed data using a histogram, density plot, or empirical CDF.

• Compare the observed shape with the model's expected shape.

• Check whether the model allows impossible values for the problem.

• Use domain knowledge; a visually similar curve does not prove the model is correct.

• Where appropriate, use diagnostic plots or formal goodness-of-fit methods.

13. Common mistakes

• Using mean alone when the data are highly skewed or contain extreme values.

• Calling every IQR-flagged point an error instead of investigating it.

• Confusing variance with standard deviation.

• Mixing sample and population variance formulas.

• Treating a histogram as a PMF: histogram bins cover intervals; a PMF assigns probabilities to discrete values.

• Assuming a continuous variable has positive probability at one exact value.

• Thinking a CDF can decrease; it cannot.

• Confusing percentile value with percentile rank.

• Assuming a random seed changes the distribution rather than making a sequence reproducible.

• Choosing a distribution based only on the shape of one small histogram.

14. Quick revision sheet

Term	One-line meaning
Distribution	Values a variable takes and how frequently/likely they occur.
Outlier	Unusual observation that needs investigation.
IQR	Q3 – Q1; spread of the middle half.

Term	One-line meaning
Variance	Average squared deviation from the mean, with population/sample convention.
PMF	Probability of each exact discrete value.
CDF	Probability that X is less than or equal to x.
Percentile	Value below which approximately a given percentage of observations fall.
Percentile rank	Relative position of a value in a reference distribution.
Class size paradox	A randomly chosen student is more likely to attend a larger class.
Normal	Symmetric bell-shaped distribution.
Exponential	Right-skewed non-negative waiting-time model.
Lognormal	Positive values whose logarithms are normally distributed.

15. Exam practice questions

1. Define a distribution and list four ways to describe it.

2. Explain how a histogram represents a distribution.

3. What is an outlier? Explain the IQR rule.

4. Differentiate mean and median when a dataset is right-skewed.

5. Define variance. Differentiate population variance and sample variance.

6. What is a PMF? State its properties and give a die example.

7. Explain the class size paradox with a numerical example.

8. Differentiate Pandas loc and iloc.

9. Define the CDF and list its important properties.

10. Differentiate PMF and CDF.

11. What is a percentile? What is a percentile rank?

12. Why is a fixed random seed useful?

13. Explain the normal distribution and the 68–95–99.7 rule.

14. Describe the exponential distribution and its mean.

15. What is a lognormal distribution? How is it related to the normal distribution?

16. Name two plots that help compare observed data with a proposed distribution model.

Short answer hints

1. Values and frequencies/probabilities; center, spread, shape, unusual values. 2. Bins show counts or density. 3. Values outside Q1−1.5IQR or Q3+1.5IQR are potential outliers. 4. Mean is pulled toward the long tail; median is more robust. 5. Population divides by N; common sample estimator divides by n−1. 6. Discrete probability; probabilities are 0–1 and sum to 1. 7. Student-based selection weights large classes more. 8. loc uses labels, iloc uses positions. 9. $F(x)=P(X\le x)$, nondecreasing, between 0 and 1. 10. PMF gives exact discrete probability; CDF accumulates probability. 11. Percentile is a cutoff value; percentile rank is a relative position. 12. Reproducible pseudo-random sequence. 13. Symmetric bell curve; about 68%, 95%, 99.7% within 1, 2, 3 standard deviations. 14. Non-negative waiting times; mean $1/\lambda$. 15. Log of the variable is normal. 16. Histogram/density plot and empirical CDF.

End of Module 4 notes. Revise the definitions and formulas, then practice explaining what each plot tells you and why a particular distribution may fit a dataset.